

EEN1035 Assignment 2: Java GUI-based Client/Server Sensor Aggregation

Name: Katherina Keogh

Student Number: 19423752

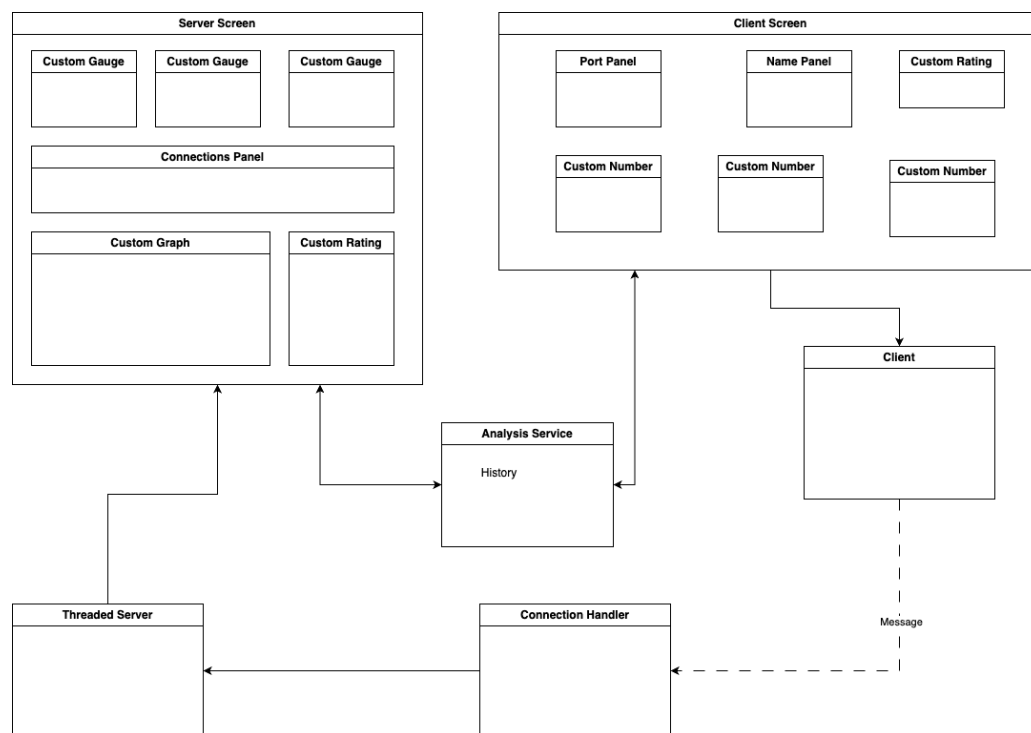
Course: MECE

1. Introduction	3
2. Design	3
2.1 Client	3
2.2 Server	4
2.3 Message	4
2.4 History	4
2.5 Analysis Service	5
3. Implementation	5
3.1. Client	5
3.2. Server	5
4. Testing	6
5. Conclusion	6
6. Appendices	7
6.1. AnalysisService.java	7
6.2. Client.java	11
6.3. ClientScreen.java	14
6.4. Constants.java	18
6.5. CustomGauge.java	19
6.6. CustomGraph.java	21
6.7. CustomNumberInput.java	23
6.8. CustomRating.java	25
6.9. History.java	26
6.10. Message.java	28
6.11. ServerScreen.java	30
6.12. ThreadedServer.java	36
6.13. ThreadedConnectionHandler.java	38

1. Introduction

The aim of my application is to display the air quality and light detected and provide users with useful insights into the outdoor readings on both the server and client sides. The server displays the average of all combined values received from clients, and the novel feature also provides a rating for outdoor conditions. The client-side displays similar information but specific to that client. The client screen displays the port as an uneditable field, the sensor name, air quality (CO₂), air pollution and light. The novel feature is also an overall rating but also specifies each environmental sensor's rating, showing the user what their individual ratings are.

2. Design



2.1 Client

The client uses a combination of Swing components to collect the simulated data from the user. Every 5 seconds `Client.java` gets the message from the Client screen, which takes the values which are displayed on screen at that moment. The Client then converts these values into a `Message` object which is then sent to the Server. It is sent to the server using the Client code provided to us but with a few tweaks.

As the Client is getting the Message data, it also updates its own local History object with the values that are being sent. The Client uses this History object to generate its rating by passing it to the Analysis Service.

The Analysis Service provides the Client Screen with metrics which are used to indicate to the user the quality of the various sensors and the overall quality.

2.2 Server

The Server is based on the code we provided and is fully threaded. It accepts any number of clients. It expects a Message object to be sent from the Client.

To dynamically accept any number of clients, I created a list of all Clients that connected to the server and a list of their 10 historical values. To do this, I converted the Message object into a History object. If the server receives multiple Messages from the Client, it will continuously update the same History object in the list. It can do so by using the number, similarly to an ID, to identify the object, as the port number is never changed.

Every 5 seconds, the Server sends the list of clients to the Analysis Service to get the metrics and display them to the user.

2.3 Message

The Message object is used to send the port number, name, status and sensor readings to the Server from the Client.

2.4 History

The History object is used to dynamically store the port, name, status and 10 most recent sensor readings.

2.5 Analysis Service

The Analysis Service is used to perform data analytics on History objects. It looks at both current readings and all historic readings. This means it can provide insightful metrics back to the Server and Clients.

3. Implementation

3.1. Client

The Client GUI was built using Java Swing and custom components. Users can input sensor values like CO2 levels, light intensity, and pollution levels, with updates shown in real-time. The novel feature includes various panels that display the current sensor ratings and use colours to highlight their status.

Each client connects to the server via sockets, sending sensor data regularly. The server identifies clients based on their port numbers and then maps it to the respective Average object. If a client disconnects, the server removes it from the active list and updates the UI accordingly. If the Server is down, the Client will try every 5 seconds to reconnect to it.

Clients also send a "final message" before disconnecting, with all readings set to zero. This ensures the server gracefully handles disconnections.

One novel feature is an overall rating, combining the CO2, light and pollution metrics for the sensor. This could be used to measure if things in agriculture, environmental or smart city applications. Another novel feature is individual ratings for CO2, light and pollution to indicate to the user if the values are potentially harmful.

3.2. Server

The Server listens for incoming client requests on a designated port, accepts connections, and assigns each connection to a separate thread for processing.

The server maintains a list of History objects that store historical sensor readings (CO2, light, and pollution) for each connected client. These objects are updated every 5 seconds and are used to calculate key metrics like averages, lowest, and highest readings. It displays

the status of connected clients, their sensor data, and calculated metrics and refreshes every 5 seconds. This interface displays custom gauges and graphs to display this data.

The novel feature provides an overall rating, combining the CO₂, light and pollution metrics from all connected sensors. This could be used to measure things in agriculture, environmental or smart city applications.

4. Testing

I carried out testing by running multiple (up to 3) Clients and the Server simultaneously. The Server was able to successfully accept up to 3 clients and display the data. Each Client was also successfully able to send Message objects to the Server.

As the values from the sensors increased, the values on the graph were more difficult to interpret. The graph also lacks units of measurement on the Y-axis.

I also validated the user input fields for the Client

- Port number was uneditable
- Name was editable and updated while running
- Sensor values could be changed by using the buttons
- Sensor vales could be change by using the keyboard and did not accept letter or negative numbers.

5. Conclusion

In conclusion, my applications implements multiple GUI client devices that send simulated environmental sensor information to a central server, where it is aggregated and displayed using a GUI. If given more time I would have liked to increase the quality of some existing GUI components, such as the custom graph. I would have also liked to be able to perform other calculations to provide even more novel features and insights to the user. I also completed this project using one laptop, and in future I would like to test using embedded devices and real data.

6. Appendices

6.1. AnalysisService.java

```
/**
 *
 * File name: AnalysisService.java
 * @author: Katherina Keogh
 * Description: This file creates the an Analysis Service which is used to
calculate
 *             metric or analyze the data.
 *
 */

package een1035;

import java.awt.Color;
import java.util.List;
import java.util.stream.Collectors;

public class AnalysisService {

    public static Integer calculateAverage(String type, List<History>
listOfClients) {
        Integer noOfClients = new Integer(listOfClients.size());
        Integer sum = new Integer(0);
        listOfClients = listOfClients.stream()
            .filter(client -> client.getStatus() != Constants.CONNECTED)
            .collect(Collectors.toList());
        if(noOfClients != 0) {
            if(type == Constants.OXYGEN) {
                for(History hist : listOfClients) {
                    sum = sum + hist.getMostRecentOxygen();
                }
                return sum / noOfClients;
            } else if (type == Constants.LIGHT) {
                for(History hist : listOfClients) {
                    sum = sum + hist.getMostRecentLight();
                }
                return sum / noOfClients;
            } else if (type == Constants.POLLUTION) {
```

```

        for(History hist : listOfClients) {
            sum = sum + hist.getMostRecentPollution();
        }
        return sum / noOfClients;
    }
}

return 0;

}

public static Integer getHighestReading(String type, List<History>
listOfClients) {
    Integer highest = new Integer(0);
    listOfClients = listOfClients.stream()
        .filter(client -> client.getStatus() != Constants.CONNECTED)
        .collect(Collectors.toList());
    if(type == Constants.OXYGEN) {
        for(History hist : listOfClients) {
            if(hist.getMostRecentOxygen() > highest) {
                highest = hist.getMostRecentOxygen();
            }
        }
    } else if (type == Constants.LIGHT) {
        for(History hist : listOfClients) {
            if(hist.getMostRecentLight() > highest) {
                highest = hist.getMostRecentLight();
            }
        }
    } else if (type == Constants.POLLUTION) {
        for(History hist : listOfClients) {
            if(hist.getMostRecentPollution() > highest) {
                highest = hist.getMostRecentPollution();
            }
        }
    }

    return highest;
}

public static Integer getLowestReading(String type, List<History>
listOfClients) {
    Integer lowest = new Integer(0);

```



```

        listOfClients = listOfClients.stream()
            .filter(client -> client.getStatus() != Constants.CONNECTED)
            .collect(Collectors.toList());
    if(listOfClients.size() > 0) {
        if(type == Constants.OXYGEN) {
            lowest = listOfClients.get(0).getMostRecentOxygen();
            for(History hist : listOfClients) {
                if(hist.getMostRecentOxygen() < lowest) {
                    lowest = hist.getMostRecentOxygen();
                }
            }
        } else if (type == Constants.LIGHT) {
            lowest = listOfClients.get(0).getMostRecentLight();
            for(History hist : listOfClients) {
                if(hist.getMostRecentLight() < lowest) {
                    lowest = hist.getMostRecentLight();
                }
            }
        } else if (type == Constants.POLLUTION) {
            lowest = listOfClients.get(0).getMostRecentPollution();
            for(History hist : listOfClients) {
                if(hist.getMostRecentPollution() < lowest) {
                    lowest = hist.getMostRecentPollution();
                }
            }
        }
    }
    return lowest;
}

public static String getRating(History average) {
    String oxygen = getOxygenRating(average.getMostRecentOxygen());
    String light = getLightRating(average.getMostRecentLight());
    String pollution = getPollutionRating(average.getMostRecentPollution());

    int rating = 0;

    if (oxygen.equals(Constants.GOOD)) {
        rating += 2;
    } else if (oxygen.equals(Constants.FAIR)) {
        rating += 1;
    }
}

```

```

        if (light.equals(Constants.GOOD)) {
            rating += 2;
        } else if (light.equals(Constants.FAIR)) {
            rating += 1;
        }

        if (pollution.equals(Constants.GOOD)) {
            rating += 2;
        } else if (pollution.equals(Constants.FAIR)) {
            rating += 1;
        }

        if (rating >= 5) {
            return Constants.GOOD;
        } else if (rating >= 3) {
            return Constants.FAIR;
        } else {
            return Constants.POOR;
        }
    }

    public static String getOxygenRating(int average) {
        if(average <= Constants.OXYGEN_GOOD) {
            return Constants.GOOD;
        } else if (average >= Constants.OXYGEN_BAD) {
            return Constants.POOR;
        } else {
            return Constants.FAIR;
        }
    }

    public static String getLightRating(int average) {
        if(average >= Constants.LIGHT_GOOD) {
            return Constants.GOOD;
        } else if (average <= Constants.LIGHT_BAD) {
            return Constants.POOR;
        } else {
            return Constants.FAIR;
        }
    }

    public static String getPollutionRating(int average) {
        if(average <= Constants.POLLUTION_NORMAL) {

```

```

        return Constants.GOOD;
    } else if (average >= Constants.POLLUTION_BAD) {
        return Constants.POOR;
    } else {
        return Constants.FAIR;
    }
}

public static Color getRatingColor(String rating) {
    if(rating == Constants.GOOD) {
        return Color.green;

    } else if (rating == Constants.FAIR) {
        return Color.YELLOW;
    } else {
        return Color.red;
    }
}
}

```

6.2. Client.java

```

/* The Client Class - Written by Derek Molloy for the EE402 Module
 * See: ee402.eeng.dcu.ie
 *
 *
 */

package een1035;

import java.net.*;
import java.io.*;

public class Client {

    private static int portNumber = 5050;
    private Socket socket = null;
    private String serverIP;
    private ObjectOutputStream os = null;
    private ObjectInputStream is = null;
    private ClientScreen clientGUI;
}

```

```

    public Client(String serverIP) {
        this.serverIP = serverIP;
        if (!connectToServer(serverIP)) {
            System.out.println("XX. Failed to open socket connection to: " +
serverIP);
        }
    }

    private boolean connectToServer(String serverIP) {
        try {
            this.socket = new Socket(serverIP, portNumber);
            this.os = new ObjectOutputStream(this.socket.getOutputStream());
            this.is = new ObjectInputStream(this.socket.getInputStream());
            System.out.println("00. -> Connected to Server:" +
this.socket.getInetAddress()
                + " on port: " + this.socket.getPort());
            System.out.println("    -> from local address: " +
this.socket.getLocalAddress()
                + " and port: " + this.socket.getLocalPort());
        }
        catch (Exception e) {
            System.out.println("XX. Failed to Connect to the Server at port: " +
portNumber);
            System.out.println("    Exception: " + e.toString());
            return false;
        }
        return true;
    }

    private void sendData() {
        Message sensorData = this.clientGUI.getMessage();
        System.out.println("01. -> Sending Message object to the server...");
        this.send(sensorData);
    }

    public void sendDisconnect(Message message) {
        System.out.println("01. -> Sending Disconnect message to the server...");
        this.send(message);
        try{
            this.socket.close();
        } catch (Exception e) {
            System.out.println("XX. Failed to Disconnect to the Server at port: "
+ portNumber);
            System.out.println("    Exception: " + e.toString());

```

```

    }

    System.out.println("04. -- Disconnected from Server.");
}

private void send(Message message) {
    try {
        System.out.println("02. -> Sending a Message...");
        message.display();
        os.writeObject(message);
        os.flush();
        System.out.println("03. -> Message sent.");
    }
    catch (Exception e) {
        System.out.println("XX. Exception Occurred on Sending:" +
e.toString());
        System.out.println("XX. Attempting to reconnect to:" + this.serverIP);
        connectToServer(this.serverIP);
    }
}

private void getGUI(Client client) {
    try {
        this.clientGUI = new ClientScreen(client, this.socket.getLocalPort());
    } catch (Exception e) {
        System.out.println("XX. Exception Occurred on opening GUI:" +
e.toString());
    }
}

public static void main(String args[])
{
    System.out.println("**. Java Client Application - EE402 OOP Module, DCU");
    if(args.length==1){
        Client theApp = new Client(args[0]);
        theApp.getGUI(theApp);
        Thread taskThread = new Thread(() -> {
            while (true) {
                try {
                    theApp.sendData();
                    Thread.sleep(5000);
                } catch (InterruptedException e) {
                    System.err.println("Task thread interrupted: " +
e.getMessage());

```

```

                break;
            }
        }
    });
    taskThread.start();

}
else
{
    System.out.println("Error: you must provide the address of the
server");
    System.out.println("Usage is:  java Client x.x.x.x  (e.g. java Client
192.168.7.2)");
    System.out.println("        or:  java Client hostname (e.g. java Client
localhost)");
}
    System.out.println("**. End of Application.");
}
}

```

6.3. ClientScreen.java

```

/**
 *
 * File: ClientScreen.java
 * Author: Katherina Keogh
 * Description: This file creates the GUI and its components for the user to
interact with the client.
 *
 */

package een1035;

import java.awt.BorderLayout;
import java.awt.Frame;
import java.awt.event.WindowEvent;
import java.awt.event.WindowListener;
import java.util.Random;

import javax.swing.Timer;

import javax.swing.JButton;
import javax.swing.JLabel;

```

```

import javax.swing.JPanel;
import javax.swing.JTextField;

@SuppressWarnings("serial")
public class ClientScreen extends Frame implements WindowListener {

    private int port;
    private JTextField nameField, portField;
    private CustomNumberInput oxygen, light, pollution;
    private JPanel north, south, rating;
    private Client client;
    private CustomRating customRating;
    private History average;

    public ClientScreen(Client client, int portNumber){
        super("Client");
        this.port = portNumber;
        this.client = client;

        this.nameField = new JTextField(String.valueOf("Name"));

        this.north = new JPanel();

        addPortDisplay();
        addNameField();
        addRating();

        this.south = new JPanel();
        addNumberFields();
        this.south.add(new JButton("Update"));

        this.add(south, BorderLayout.SOUTH);
        this.add(north, BorderLayout.NORTH);

        this.addWindowListener(this);
        this.pack();
        this.setVisible(true);

        this.average = new History(0, "Average", "N/A", 0, 0, 0);
        int i = 0;
    }
}

```

```

        while(i < 10) {
            i++;
            this.average.addOxygen(0);
            this.average.addLight(0);
            this.average.addPollution(0);

        }

        refresh();
    }

    private void refresh() {

        Timer timer = new Timer(5000, e -> {
            updateComponents();

        });
        timer.start();

    }

    private void updateComponents() {
        this.customRating.updateRating(AnalysisService.getRating(this.average));
    }

    private void addPortDisplay() {
        JPanel portPanel = new JPanel();
        this.portField = new JTextField(String.valueOf(this.port));
        portField.setEditable(false);
        portPanel.add(new JLabel("Port:"), BorderLayout.NORTH);
        portPanel.add(portField);

        north.add(portPanel);

    }

    private void addNameField() {
        JPanel namePanel = new JPanel();
        this.nameField = new JTextField("Senor on port " +
String.valueOf(this.port));
        namePanel.add(new JLabel("Name:"), BorderLayout.NORTH);
        namePanel.add(nameField);

        north.add(namePanel);
    }

```



```

    }

    private void addNumberFields() {
        this.oxygen = new CustomNumberInput(Constants.OXYGEN);
        this.light = new CustomNumberInput(Constants.LIGHT);
        this.pollution = new CustomNumberInput(Constants.POLLUTION);
        south.add(this.oxygen, BorderLayout.CENTER);
        south.add(new JLabel("ppm"));
        south.add(this.light, BorderLayout.CENTER);
        south.add(new JLabel("lux"));
        south.add(this.pollution, BorderLayout.CENTER);
        south.add(new JLabel("ppm"));
    }

    private void addRating() {
        this.rating = new JPanel();
        this.customRating = new CustomRating();

        this.rating.add(this.customRating);
        this.north.add(this.rating);
    }

    public Message getMessage() {
        // random number to simulate noise
        Random rand = new Random();

        Message msg = new Message(this.nameField.getText(), Constants.CONNECTED,
this.port, (this.oxygen.getValue() + rand.nextInt(100)), (this.light.getValue()
+ rand.nextInt(100)), (this.pollution.getValue() + rand.nextInt(100)));

        // before message is sent, add the values to local history for data
analysis
        this.average.addLight(msg.getLight());
        this.average.addOxygen(msg.getOxygen());
        this.average.addPollution(msg.getPollution());
        return msg;
    }

    private void sendDisconnect() {
        System.out.println("DISCONNECTNG " + this.nameField.getText() + ":" +
this.port + ".....");
        this.client.sendDisconnect(new Message(this.nameField.getText(),
Constants.DISCONNECTED, this.port, 0, 0,0));
        System.exit(0);
    }

```

```

    }

    public void windowActivated(WindowEvent arg0) {}
    public void windowClosed(WindowEvent arg0) {}
    public void windowClosing(WindowEvent arg0) { sendDisconnect(); }
    public void windowDeactivated(WindowEvent arg0) {}
    public void windowDeiconified(WindowEvent arg0) {}
    public void windowIconified(WindowEvent arg0) {}
    public void windowOpened(WindowEvent arg0) {}
}

```

6.4. Constants.java

```

/**
 *
 * File: Constants.java
 * Author: Katherina Keogh
 * Description: This file creates the constants which are used through the
application.
 *
 */

package een1035;

public class Constants {

    public static final String OXYGEN = "CO2";
    public static final String LIGHT = "Light";
    public static final String POLLUTION = "Pollution";

    public static final String CONNECTED = "connected";
    public static final String DISCONNECTED = "disonnected";

    public static final Integer OXYGEN_GOOD = 4000;
    public static final Integer OXYGEN_NORMAL = 20000;
    public static final Integer OXYGEN_BAD = 50000;

    public static final Integer LIGHT_GOOD = 50000;
    public static final Integer LIGHT_NORMAL = 25000;
    public static final Integer LIGHT_BAD = 1000;
}

```

```

    public static final Integer POLLUTION_GOOD = 0;
    public static final Integer POLLUTION_NORMAL = 25000;
    public static final Integer POLLUTION_BAD = 50000;

    public static final String GOOD = "Good";
    public static final String FAIR = "Fair";
    public static final String POOR = "Poor";
}

```

6.5. CustomGauge.java

```

/**
 *
 * File name: CustomGauge.java
 * @author: Katherina Keogh
 * Description: This file creates the a custom Gauge component for the GUI.
 *              The gauge displays the lowest, average and highest readings
 *              from the sensors and plots them on a bar.
 *
 * Reference:
https://docs.oracle.com/javase/1.5.0/docs/guide/awt/demos/lightweight/GaugeSwing/Gauge.java
 *
 */

package een1035;

import java.awt.Color;
import java.awt.Dimension;
import java.awt.Font;
import java.awt.FontMetrics;
import java.awt.Graphics;
import javax.swing.*;

public class CustomGauge extends JPanel {

    private int lowValue = 0;
    private int avgValue = 0;
    private int highValue = 0;
    int current = 0;

```

```

int total = 50000;

int Height = 18;
int Width = 250;

public void setValues(int low, int avg, int high) {
    this.lowValue = low;
    this.avgValue = avg;
    this.highValue = high;
    repaint();
}

public CustomGauge(int min, int avg, int max) {
    this(Color.lightGray);
    setValues(min, avg, max);
    this.setPreferredSize(new Dimension(200, 120));
}

public CustomGauge(Color gaugeColor) {
    setBackground(gaugeColor);
}

public void paint(Graphics g) {
    int barWidth = (int) (((float)this.highValue/(float)total) *
    getSize().width);
    g.setColor(new java.awt.Color(74, 176, 240));
    g.fill3DRect(0, 0, barWidth, getSize().height-2, true);

    barWidth = (int) (((float)this.avgValue/(float)total) *
    getSize().width);
    g.setColor(new java.awt.Color(141, 206, 247));
    g.fill3DRect(0, 0, barWidth, getSize().height-2, true);

    barWidth = (int) (((float)this.lowValue/(float)total) *
    getSize().width);
    g.setColor(new java.awt.Color(187, 226, 250));
    g.fill3DRect(0, 0, barWidth, getSize().height-2, true);
}

public Dimension getPreferredSize() {
    return new Dimension(Width, Height);
}

```

```
}
```

6.6. CustomGraph.java

```
/**
 *
 * File name: CustomGraph.java
 * @author: Katherina Keogh
 * Description: This file creates the a custom Graph component for the GUI.
 *             The graph displays the average of the last 10 historical readings
 *             from the sensors and plots a graph.
 * Reference: https://www.javatpoint.com/java-plot
 */

package een1035;

import java.awt.*;
import javax.swing.*;

public class CustomGraph extends JPanel {

    // set the sensor values to be 0 to begin
    Integer[] oxygen = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0};
    Integer[] light = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0};
    Integer[] pollution = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0};
    Integer start = new Integer (0);

    public CustomGraph(){

    }

    public void updateValues(Integer[] oxygen, Integer[] light, Integer[]
pollution) {
        this.oxygen = oxygen;
        this.light = light;
        this.pollution = pollution;
        start = start + 5;
        repaint();
    }

    @Override
    protected void paintComponent(Graphics g) {
```

```

        super.paintComponent(g);
        Graphics2D g2d = (Graphics2D) g;

        g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);

        int width = getWidth();
        int height = getHeight();

        g2d.setColor(Color.BLACK);
        g2d.setStroke(new BasicStroke(5));
        g2d.drawLine(0, height - 30, 360, height - 30); // x axis line
        g2d.drawLine(30, 0, 30, height - 30 ); // y axis line

        drawLabels(g2d, width - 30, height - 10); // adds the time scale to x axis
        drawPoints(g2d, width - 30, height - 30, this.oxygen, Color.CYAN);
        drawPoints(g2d, width - 30, height - 30, this.light, Color.YELLOW);
        drawPoints(g2d, width - 30, height - 30, this.pollution, Color.GREEN);
    }

    private void drawLabels(Graphics2D g2d, int x, int y) {
        int step = x / 10; // gets the amount to increase by, based on teh size of
the graph

        int point = 30;
        int display = this.start;

        while(point <= x + 25 ) {
            g2d.setFont(new Font("Arial", Font.PLAIN, 14));
            g2d.drawString(String.valueOf(display), point, y);
            point = point + step;
            display += 5;
        }
    }

    private void drawPoints(Graphics2D g2d, int x, int y, Integer[] points, Color
color) {
        int yStep = y / 5;
        int xStep = x / 10;
        int point = 30;

        for (int i = 0; i < points.length; i++) {

```

```

        int value = points[i];
        int graphY = y - (value * y / 55000); // scales it down

        g2d.setColor(color); // adds the colour based on sensor data (cyan,
green or yellow)
        g2d.fillOval(point, graphY - 10, 10, 10);

        g2d.setFont(new Font("Arial", Font.PLAIN, 12));
        g2d.setColor(Color.black);
        g2d.drawString(String.valueOf(value), point - 10, graphY - 10); //
adds the value as a lable

        point += xStep;
    }
}
}

```

6.7. CustomNumberInput.java

```

/**
 *
 * File name: CustomNumberInput.java
 * @author: Katherina Keogh
 * Description: This file creates the a custom Number Input component for the
GUI.
 *
 * The number input creates an editable field for users to input the
 * sensor value using either the keyboard or the buttons displayed
 * on screen.
 *
 */

package een1035;
import javax.swing.*.*;
import javax.swing.event.ChangeEvent;
import javax.swing.event.ChangeListener;

import java.awt.*.*;

@SuppressWarnings("serial")
public class CustomNumberInput extends JPanel {

```

```

private JSpinner numberInput;

public CustomNumberInput(String label) {

    this.setLayout(new FlowLayout());
    numberInput = new JSpinner(new SpinnerNumberModel(0, 0, 100000, 1));
    JSpinner.NumberEditor editor = new JSpinner.NumberEditor(this.numberInput,
"#");
    numberInput.setEditor(editor);
    this.add(new JLabel(label + ":"), BorderLayout.NORTH);
    this.setBackground(getBackgroundColour(label));
    this.add(numberInput);

    numberInput.addChangeListener(new ChangeListener() {
        @Override
        public void stateChanged(ChangeEvent e) {
            setBackground(getBackgroundColour(label));
        }
    });
}

public int getValue() {
    return (int) numberInput.getValue();
}

private Color getBackgroundColour(String label){
    if(label == Constants.OXYGEN){
        return
AnalysisService.getRatingColor(AnalysisService.getOxygenRating((int)
numberInput.getValue()));
    } else if (label == Constants.LIGHT) {
        return
AnalysisService.getRatingColor(AnalysisService.getLightRating((int)
numberInput.getValue()));
    } else if (label == Constants.POLLUTION){
        return
AnalysisService.getRatingColor(AnalysisService.getPollutionRating((int)
numberInput.getValue()));
    }
    return Color.lightGray;
}
}

```


6.8. CustomRating.java

```
/**
 *
 * File name: CustomRating.java
 * @author: Katherina Keogh
 * Description: This file creates the a custom Rating component for the GUI.
 *              The panel displays "Good", "Fair" or "Poor" and their
corresponding colours.
 *
 */

package een1035;

import java.awt.BorderLayout;

import javax.swing.JLabel;
import javax.swing.JPanel;

public class CustomRating extends JPanel {

    private JLabel ratingLabel;

    public CustomRating(){

        this.setLayout(new BorderLayout());

        ratingLabel = new JLabel("Rating: N/A");
        this.add(ratingLabel, BorderLayout.CENTER);

    }

    public void updateRating(String rating) {
        ratingLabel.setText("Rating: " + rating);
        this.setBackground(AnalysisService .getRatingColor(rating));
    }
}
```

6.9. History.java

```
/**
 *
 * File name: History.java
 * @author: Katherina Keogh
 * Description: This file creates the a History class which is used to store
 *             the historical data.
 */

package een1035;

public class History {

    private static final int MAX_SIZE = 10;
    private int port;
    private String name, status;
    private Integer[] oxygen = new Integer[MAX_SIZE], light = new
Integer[MAX_SIZE], pollution = new Integer[MAX_SIZE];

    public History(int port, String status, String name, Integer oxygen, Integer
light, Integer pollution) {
        this.port = port;
        this.status = status;
        this.name = name;
        addOxygen(oxygen);
        addLight(light);
        addPollution(pollution);
    }

    public int getPort() {
        return this.port;
    }

    public String getName() {
        return this.name;
    }

    public String getStatus() {
        return this.status;
    }

    public Integer[] getAllOxygen() {
        return this.oxygen;
    }
}
```

```
public Integer[] getAllLight() {
    return this.light;
}

public Integer[] getAllPollution() {
    return this.pollution;
}

public Integer getMostRecentOxygen() {
    return this.oxygen[this.oxygen.length - 1];
}

public Integer getMostRecentLight() {
    return this.light[this.light.length - 1];
}

public Integer getMostRecentPollution() {
    return this.pollution[this.pollution.length - 1];
}

public void setName(String name) {
    this.name = name;
}

public void setStatus(String status) {
    this.status = status;
}

public void addOxygen(Integer newValue) {
    this.oxygen = shiftList(this.oxygen);
    this.oxygen[this.oxygen.length-1] = new Integer(newValue);
}

public void addLight(Integer newValue) {
    this.light = shiftList(this.light);
    this.light[this.light.length-1] = new Integer(newValue);
}

public void addPollution(Integer newValue) {
    this.pollution = shiftList(this.pollution);
    this.pollution[this.pollution.length-1] = new Integer(newValue);
}

private String displayList(Integer[] list) {
```

```

        String stringOfInts = new String();
        for (int i = 0; i < list.length - 1; i++) {
            if(list[i] == null) {
                i = 0;
            }
            stringOfInts = stringOfInts + list[i] + " ";
        }
        return stringOfInts;
    }

    private Integer[] shiftList(Integer[] intList) {
        for (int i = 0; i < intList.length - 1; i++) {
            intList[i] = intList[i + 1];
        }
        return intList;
    }

    public void display() {
        System.out.println("Average{" +
            "port=" + port +
            ", name=" + name +
            ", status=" + status +
            ", oxygen=" + displayList(this.oxygen) +
            ", light=" + displayList(this.light) +
            ", pollution=" + displayList(this.pollution) +
            '}');
    }
}

```

6.10. Message.java

```

/**
 *
 * File name: Message.java
 * @author: Katherina Keogh
 * Description: This file creates the a Message class which is used to create
 *              an object which is sent from the Client to the Server.
 *
 */

```

```
package een1035;

import java.io.Serializable;

@SuppressWarnings("serial")
public class Message implements Serializable {

    private String name, status;
    private int port, oxygen, light, pollution;

    public Message(String name, String status, int port, int oxygen, int light,
int pollution) {
        this.name = name;
        this.status = status;
        this.port = port;
        this.oxygen = oxygen;
        this.light = light;
        this.pollution = pollution;
    }

    public String getName() {
        return name;
    }

    public String getStatus() {
        return status;
    }

    public void setName(String name) {
        this.name = name;
    }

    public int getPort() {
        return port;
    }

    public int getOxygen() {
        return oxygen;
    }

    public int getLight() {
        return light;
    }

    public int getPollution() {
```

```

        return pollution;
    }

    public void display() {
        System.out.println("Message {" +
            "port=" + port +
            ", name=" + name +
            ", status=" + status +
            ", oxygen=" + oxygen +
            ", light=" + light +
            ", pollution=" + light +
            '}');
    }
}

```

6.11. ServerScreen.java

```

/**
 *
 * File: ServerScreen.java
 * Author: Katherina Keogh
 * Description: This file creates the GUI and its components for the user to
interact with the server.
 *
 */

package een1035;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.Frame;
import java.awt.event.WindowEvent;
import java.awt.event.WindowListener;
import java.util.ArrayList;
import java.util.List;

import javax.swing.BorderFactory;
import javax.swing.BoxLayout;

```

```

import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.JTable;
import javax.swing.table.DefaultTableModel;

@SuppressWarnings("serial")
public class ServerScreen extends Frame implements WindowListener {

    private JPanel oxygenPanel, lightPanel, pollutionPanel, graph, rating, list,
connectionsPanel;

    private CustomGraph customGraph;
    private CustomGauge oxygenGauge, lightGauge, pollutionGauge ;
    private JLabel connectionsLabel;
    private List<History> listOfClients = new ArrayList<History>();
// private JTable clientTable;
// private DefaultTableModel tableModel;
    private History averageHist;
    private CustomRating customRating;

    public ServerScreen(){
        super("Server");

        // initialise averageHist with all 0s since no data has been recieved
        this.averageHist = new History(0, "Average", "N/A", 0, 0, 0);
        int i = 0;
        while(i < 10) {
            i++;
            this.averageHist.addOxygen(0);
            this.averageHist.addLight(0);
            this.averageHist.addPollution(0);
        }

        // creating the components
        createOxygenPanel();
        createLightPanel();
        createPollutionPanel();
        createConnectionsPanel();
        createCustomGraphPanel();
        createRatingPanel();

        // creating the north panel
        JPanel north = new JPanel();
        north.add(this.oxygenPanel);
        north.add(this.lightPanel);
        north.add(this.pollutionPanel);

```

```

        // creating the south pane;
        JPanel south = new JPanel();
        south.add(this.graph);

        //creating the centre panel
        JPanel centre = new JPanel();
        centre.setLayout(new BorderLayout(centre, BorderLayout.Y_AXIS));
        centre.add(this.rating);
        centre.add(this.connectionsPanel);

        // adding panels to frame
        this.add(south, BorderLayout.SOUTH);
        this.add(north, BorderLayout.NORTH);
        this.add(centre, BorderLayout.CENTER);

        this.addWindowListener(this);
        this.pack();
        this.setVisible(true);

        // loop to update values every 5 seconds
        Thread taskThread = new Thread(() -> {
            while (true) {
                try {
                    updateComponents();
                    Thread.sleep(5000);
                } catch (InterruptedException e) {
                    System.err.println("Task thread interrupted: " +
e.getMessage());
                    break;
                }
            }
        });
        taskThread.start();
    }

    public void createCustomGraphPanel() {
        this.graph = new JPanel();
        this.customGraph = new CustomGraph();
        this.graph.setPreferredSize(new Dimension(380, 300));
        this.graph.add(this.customGraph, BorderLayout.NORTH);
        this.graph.setLayout(new BorderLayout(this.graph, BorderLayout.Y_AXIS));
    }

```



```

private void createOxygenPanel() {
    this.oxygenPanel = new JPanel();
    this.oxygenPanel.add(new JLabel(Constants.OXYGEN));
    this.oxygenGauge = new
CustomGauge(AnalysisService.getLowestReading(Constants.OXYGEN,
this.listOfClients), AnalysisService.calculateAverage(Constants.OXYGEN,
this.listOfClients), AnalysisService.getHighestReading(Constants.OXYGEN,
this.listOfClients));
    this.oxygenPanel.add(this.oxygenGauge, BorderLayout.CENTER);
}

private void createLightPanel() {
    this.lightPanel = new JPanel();
    this.lightPanel.add(new JLabel(Constants.LIGHT));
    this.lightGauge = new
CustomGauge(AnalysisService.getLowestReading(Constants.LIGHT,
this.listOfClients), AnalysisService.calculateAverage(Constants.LIGHT,
this.listOfClients), AnalysisService.getHighestReading(Constants.LIGHT,
this.listOfClients));
    this.lightPanel.add(this.lightGauge);
}

private void createPollutionPanel() {
    this.pollutionPanel = new JPanel();
    this.pollutionPanel.add(new JLabel(Constants.POLLUTION));
    this.pollutionGauge = new
CustomGauge(AnalysisService.getLowestReading(Constants.POLLUTION,
this.listOfClients), AnalysisService.calculateAverage(Constants.POLLUTION,
this.listOfClients), AnalysisService.getHighestReading(Constants.POLLUTION,
this.listOfClients));
    this.pollutionPanel.add(this.pollutionGauge);
}

private void createRatingPanel() {
    this.rating = new JPanel();
    this.customRating = new CustomRating();
    this.customRating.setBorder(BorderFactory.createLineBorder(Color.BLACK,
1));
    this.rating.add(this.customRating);
}

private void createConnectionsPanel() {
    this.connectionsPanel = new JPanel();

```

```

//      String[] columnNames = {"Name", "Port Number"};
//
//      this.tableModel = new DefaultTableModel(columnNames, 0);
//
//      this.clientTable = new JTable(tableModel);
//      this.connectionsPanel.add(clientTable);

this.connectionsPanel = new JPanel();
this.connectionsLabel = new JLabel("No devices have connected yet.");

this.connectionsPanel.setBorder(BorderFactory.createLineBorder(Color.BLACK, 1));

this.connectionsPanel.add(this.connectionsLabel);

}

// sets the new values when the server recieves a message
public void setValues(Message message) {

    // first checks if the list of clients contains the client that sent the
message.
    // using the port kind of like an ID since it changed be chnaged while
running
    if(checkIfListContains(message.getPort())) {
        // if it is in the list then add values to the existing object
        History currentAvg = this.listOfClients.stream()
            .filter(client -> client.getPort() == message.getPort())
            .findFirst()
            .orElse(null);

        currentAvg.setName(message.getName());
        currentAvg.setStatus(message.getStatus());
        currentAvg.addOxygen(message.getOxygen());
        currentAvg.addLight(message.getLight());
        currentAvg.addPollution(message.getPollution());
    } else {
        // if its not in the list then create a new object
        this.listOfClients.add(new History(message.getPort(),
message.getStatus(), message.getName(), new Integer(message.getOxygen()), new
Integer(message.getLight()), new Integer(message.getPollution())));
    }
}

private void updateComponents() {
    this.averageHist.display();
}

```

```

this.customRating.updateRating(AnalysisService.getRating(this.averageHist));
    displayAverage();
    displayConnections();

this.oxygenGauge.setValues(AnalysisService.getLowestReading(Constants.OXYGEN,
this.listOfClients), AnalysisService.calculateAverage(Constants.OXYGEN,
this.listOfClients), AnalysisService.getHighestReading(Constants.OXYGEN,
this.listOfClients));

this.lightGauge.setValues(AnalysisService.getLowestReading(Constants.LIGHT,
this.listOfClients), AnalysisService.calculateAverage(Constants.LIGHT,
this.listOfClients), AnalysisService.getHighestReading(Constants.LIGHT,
this.listOfClients));

this.pollutionGauge.setValues(AnalysisService.getLowestReading(Constants.POLLUTIO
N, this.listOfClients), AnalysisService.calculateAverage(Constants.POLLUTION,
this.listOfClients), AnalysisService.getHighestReading(Constants.POLLUTION,
this.listOfClients));

};

private Boolean checkIfListContains(Integer port) {
    for (History avg : this.listOfClients) {
        if (avg.getPort() == port) {
            return true;
        }
    }
    return false;
}

private void displayAverage() {
    if(this.listOfClients.size() > 0) {

this.averageHist.addOxygen(AnalysisService.calculateAverage(Constants.OXYGEN,
this.listOfClients));

this.averageHist.addLight(AnalysisService.calculateAverage(Constants.LIGHT,
this.listOfClients));

this.averageHist.addPollution(AnalysisService.calculateAverage(Constants.POLLUTIO
N, this.listOfClients));

```

```

this.customGraph.updateValues(this.averageHist.getAllOxygen(),this.averageHist.ge
tAllLight(), this.averageHist.getAllPollution() );
    }

}

private void displayConnections() {
    if(this.listOfClients.size() > 0) {
        String listOfCons = "<html>";
        for(History client : listOfClients) {

            listOfCons = listOfCons + client.getName() + ":" +
client.getPort() + " " + client.getStatus().toLowerCase() + "<br/>";
        }
        listOfCons = listOfCons + "</html>";

        this.connectionsLabel.setText(listOfCons);
    }
    // I tried to implement a table but it was quite challanging to make it
dynamic
//
//     for(History client : listOfClients) {
//         this.tableModel.addRow(new Object[]{client.getName(),
client.getPort()});
//     }
}

public void windowActivated(WindowEvent arg0) {}
public void windowClosed(WindowEvent arg0) {}
public void windowClosing(WindowEvent arg0) { System.exit(0); }
public void windowDeactivated(WindowEvent arg0) {}
public void windowDeiconified(WindowEvent arg0) {}
public void windowIconified(WindowEvent arg0) {}
public void windowOpened(WindowEvent arg0) {}
}

```

6.12. ThreadedServer.java

```

/* The Date Server Class - Written by Derek Molloy for the EE402 Module
* See: ee402.eeng.dcu.ie

```

```

*/

package een1035;

import java.net.*;
import java.io.*;

public class ThreadedServer
{
    private static int portNumber = 5050;

    public static void main(String args[]) {

        boolean listening = true;
        ServerSocket serverSocket = null;

        ServerScreen serverGUI = new ServerScreen();

        try
        {
            serverSocket = new ServerSocket(portNumber);
            System.out.println("New Server has started listening on port: " +
portNumber );
        }
        catch (IOException e)
        {
            System.out.println("Cannot listen on port: " + portNumber + ",
Exception: " + e);
            System.exit(1);
        }

        while (listening)
        {
            Socket clientSocket = null;
            try{
                System.out.println("**. Listening for a connection...");
                clientSocket = serverSocket.accept();
                System.out.println("00. <- Accepted socket connection from a
client: ");

                System.out.println("    <- with address: " +
clientSocket.getInetAddress().toString());

                System.out.println("    <- and port number: " +
clientSocket.getPort());
            }

```

```

        catch (IOException e){
            System.out.println("XX. Accept failed: " + portNumber + e);
            listening = false;
        }

        ThreadedConnectionHandler con = new
ThreadedConnectionHandler(clientSocket, serverGUI);
        con.start();
        System.out.println("02. -- Finished communicating with client:" +
clientSocket.getInetAddress().toString());
    }

    try
    {
        System.out.println("04. -- Closing down the server socket
gracefully.");
        serverSocket.close();
    }
    catch (IOException e)
    {
        System.err.println("XX. Could not close server socket. " +
e.getMessage());
    }
}
}

```

6.13. ThreadedConnectionHandler.java

```

/* The Connection Handler Class - Written by Derek Molloy for the EE402 Module
 * See: ee402.eeng.dcu.ie
 */

package een1035;

import java.net.*;
import java.io.*;

public class ThreadedConnectionHandler extends Thread
{
    private Socket clientSocket = null;
    private ObjectInputStream is = null;
    private ObjectOutputStream os = null;
    private ServerScreen serverGUI;
}

```

```

    public ThreadedConnectionHandler(Socket clientSocket, ServerScreen serverGUI)
    {
        this.clientSocket = clientSocket;
        this.serverGUI = serverGUI;
    }

    public void run() {
        try {
            this.is = new ObjectInputStream(clientSocket.getInputStream());
            this.os = new ObjectOutputStream(clientSocket.getOutputStream());
            while (this.readMessage()) {}
        }
        catch (IOException e)
        {
            System.out.println("XX. There was a problem with the Input/Output
Communication:");
            e.printStackTrace();
        }
    }

    private boolean readMessage() {
        Message s = null;
        try {
            s = (Message) is.readObject();
        }
        catch (Exception e){
            this.closeSocket();
            return false;
        }
        this.serverGUI.setValues(s);

        return true;
    }

    public void closeSocket() { //gracefully close the socket connection
        try {
            this.os.close();
            this.is.close();
            this.clientSocket.close();
        }
        catch (Exception e) {
            System.out.println("XX. " + e.getStackTrace());
        }
    }
}

```

}